HACKER TECHNIQUES TOOLS AND INCIDENT HANDLING

A PROJECT REPORT

SSRF Testing Tool

Roll No: 727823TUCY014

Name: Harini P

1. Tool Description

The SSRF Testing Tool is a Python-based security assessment framework designed to detect Server-Side Request Forgery (SSRF) vulnerabilities in web applications. SSRF allows an attacker to trick a server into making HTTP requests to internal services, cloud metadata endpoints, or sensitive files that are inaccessible from outside the network.

The tool was built and tested in an isolated VirtualBox lab running Kali Linux, with a custom Flask vulnerable server as the target. It injects 25+ payloads across 5 categories and classifies each response as VULNERABLE, POSSIBLE, FILTERED, or BLOCKED.

Key Features

- 5 payload categories: localhost bypass, cloud metadata, internal network, protocol abuse, filter bypass
- Heuristic response analysis to detect SSRF patterns
- JSON output saved with timestamps for every test run
- 3-stage pipeline: setup_lab.py → run_tool.py → analyze_results.py

Technologies: Python 3.11, requests, flask, VirtualBox, Kali Linux

2. Test Results

Three distinct test cases were run against http://127.0.0.1:5000/fetch.

#	Category	Payload	Status	Verdict	Time (ms)
1	Localhost Bypass	http://127.0.0.1	500	NOT_VULNERABLE	5.29
1	Localhost Bypass	http://localhost	500	NOT_VULNERABLE	3.65
1	Localhost Bypass	http://0.0.0.0	500	NOT_VULNERABLE	3.37
2	Cloud Metadata	http://169.254.169.254/...	500	NOT_VULNERABLE	5.74

#	Category	Payload	Status	Verdict	Time (ms)
2	Cloud Metadata	http://metadata.google.internal/...	500	VULNERABLE	93.64
3	Protocol Abuse	file:///etc/passwd	500	VULNERABLE	4.28
3	Protocol Abuse	file:///etc/hosts	500	NOT_VULNERABLE	2.30
3	Protocol Abuse	dict://localhost:11211/stats	500	NOT_VULNERABLE	1.45

Analysis: Test Case 1 confirmed the SSRF injection vector is active — the server processed all payloads before returning 500. Test Case 2 found the Google Cloud metadata endpoint VULNERABLE with 93.64ms response time, indicating a real DNS resolution attempt. Test Case 3 confirmed file:///etc/passwd as VULNERABLE, showing the server attempted local filesystem access.

3. Real Error Encountered

While running run_tool.py, the following error occurred:

```
File "code/run_tool.py", line 9
  ROLL_NUMBER = 727823TUCY014
                ^
SyntaxError: invalid decimal literal
```

Python tried to parse 727823TUCY014 as a number but failed because it contains letters. The fix was adding quotes to make it a string:

python

```
ROLL_NUMBER = "727823TUCY014"
```

This fix was applied to all four pipeline scripts.

4. Ethical Considerations

All testing was performed exclusively on student-owned systems inside an isolated VirtualBox VM with no internet access. The Flask target server was purpose-built for this lab and never exposed to any real network. No external systems were targeted.

SSRF knowledge from this assignment is used purely for defensive purposes — to help developers implement proper URL validation, allowlists, and protocol restrictions in real applications.

- All exploitation done on isolated VMs only
- No real credentials or systems were compromised
- All scripts include FOR LAB USE ONLY warnings